



UNIVERSITÀ DI PISA

Data Science and Business Informatics

Optimization for Data Science

Project Report

Candidati:

Emiliano Marrale 545552
Salvatore Puccio 547518

ANNO ACCADEMICO 2025/2026

Contents

1	Introduction	3
2	Problem Characterization	3
2.1	Convex Program	3
2.2	Disjoint Simplices	3
2.2.1	Feasible Set	4
2.3	Conditional Gradient Method	4
2.3.1	Linear Problem	5
2.3.2	Frank-Wolfe Gap	5
2.3.3	Convergence Analysis	6
3	Data Characterization	7
3.1	Frank–Wolfe Data Generation	8
3.1.1	Matrix Generation	8
3.1.2	Characterization of the Linear Term	8
3.2	Dataset Sizes	10
4	Conditional Gradient Implementation	10
4.1	Code Snippets	10
4.2	Convergence And Evaluation	10
5	General-Purpose Solver Implementation (A2)	10
5.1	Code Snippets	11
5.2	Convergence And Evaluation	11
6	Conclusions	11

1 Introduction

(P) is the convex quadratic program on a set of K disjoint simplices

$$\min \{x^T Q x + q x : \sum_{i \in I^k} x_i = 1, k \in K, x \geq 0\}$$

where $x \in \mathbb{R}^n$, the index sets I^k form a partition of the set $\{1, \dots, n\}$ (i.e. $\cup_{k \in K} I^k = \{1, \dots, n\}$, and $I^h \cap I^k = \emptyset$ for all h and k), and Q is an $n \times n$ Positive Semidefinite matrix.

The objective of the project is to solve the problem described above using a (A1) conditional gradient method (Frank-Wolfe Type) and (A2) a general-purpose solver applied to an appropriate formulation of the problem. No off-the-shelf solvers allowed, save for (A2): the LP within the algorithm must be solved by an ad-hoc approach exploiting the structure of the problem.

2 Problem Characterization

In this section we are going to characterize all the different mathematical concepts and related implications to the problem.

2.1 Convex Program

Our objective function $f(x) = x^T Q x + q x$ is a quadratic form. Since Q is defined as Positive Semidefinite (PSD) (as stated in the project assignment), the curvature of the function is always "upward". If Q has zero eigenvalues, the function remains convex but may exhibit flat directions, and the minimizer over the feasible set does not need to be unique.

2.2 Disjoint Simplices

The feasible region is a collection of simplices. Simplices are convex sets, and their intersection (or in this case, their product space) is also convex. In our problem, the variable vector x is partitioned into groups based on the index sets I^k . Each group of variables must sum to 1 and be non-negative. The "disjoint" nature implies that the constraints are block-separable. Variables in I^1 do not affect the sum of variables in I^2 . We have a very sparse structure. This will allow us to decompose the problem during the "Linear Subproblem" phase of the Frank-Wolfe algorithm.

2.2.1 Feasible Set

For a fixed k , the constraint would be:

$$\sum_{i \in I^k} x_i = 1, x_i \geq 0$$

which is the definition of a simplex. So if $|I^k| = m$ then $(x_i)_{i \in I^k}$ lives in an $(m - 1)$ -dimensional simplex and since our I^k 's are disjoint, the full **feasible set** is of the form:

$$\mathcal{D} = \Delta_{I^1} \times \dots \times \Delta_{I^{|k|}}$$

Thus the feasible set is convex, polyhedral and will have extreme points (vertices). An **extreme point** x in a convex set $\mathcal{C}$ is extreme if it cannot be written as a non-trivial convex combination of two different points in $\mathcal{C}$:

$$x = \lambda y + (1 - \lambda)z, \lambda \in (0, 1), y, z \in \mathcal{C} \implies y = z = x$$

2.3 Conditional Gradient Method

The Conditional Gradient method is an iterative first-order optimization algorithm. Unlike Gradient Descent, which requires a "projection" step to stay inside the feasible region, Frank-Wolfe stays inside the region by solving a linear approximation. The steps of the algorithm involves:

- Start with a feasible point $x^{(t)}$.
- (Direction Finding) Find a point $s^{(t)}$ in the feasible region that minimizes the linear approximation. This is the linear problem described by the formula below:

$$s^{(t)} = \min_{s \in \mathcal{D}} \langle \nabla f(x^{(t)}), s \rangle$$

.

- (Update) Move x toward s : $x^{(t+1)} = x^{(t)} + \alpha(s^{(t)} - x^{(t)})$, where $\alpha \in [0, 1]$ is the step size.

The step size α can either be set fixed as $\alpha = \frac{2}{k+2}$ or dynamically by finding the α that minimizes $f(x_k) + \alpha(s_k - x_k)$ with $\alpha \in [0, 1]$ (that is the line search). Since our problem is quadratic, we will be using **Exact Line Search** in closed form that yields the following minimizer:

$$a^* = \begin{cases} \min\{1, \max\{0, -b/2a\}\}, & a > 0 \\ 1, & a = 0 \end{cases}$$

where $a = d_k^T Q d_k \geq 0$, $b = 2d_k^T Q x_k + q^T d_k$ and $d_k = s^t - x^t$ (Frank-Wolfe Direction)

2.3.1 Linear Problem

In each iteration of the algorithm, we must solve the **Linear Minimization Oracle** problem:

$$\min_{s \in \mathcal{D}} \langle \nabla f(x^{(t)}), s \rangle$$

Because our set K is a collection of disjoint simplices, this minimization decomposes by block as:

$$\langle \nabla f(x^t), s \rangle = \sum_{k \in K} \sum_{i \in I^k} (\nabla f(x^t))_i s_i$$

Basically, for each simplex $k \in K$, we are solving:

$$\min \sum_{i \in I^k} (\nabla f(x^{(t)}))_i \cdot s_i \quad \text{subject to} \quad \sum_{i \in I^k} s_i = 1, s_i \geq 0$$

The solution to a linear objective over a standard simplex is simply a unit vector (an extreme point).

In other words, for each index set I^k , we are going to find the index i^* that has the smallest (most negative) gradient component. Then we are going to set $s_{i^*} = 1$ and all other $s_i = 0$ for that block. This allows us to reduce a complex optimization subproblem to a simple "find the minimum" search across the gradient vector for each partition.

2.3.2 Frank-Wolfe Gap

Since the problem is convex, the first-order Taylor approximation of f at the current iterate $x^{(t)}$ is a global lower bound on f . Minimizing this linear model over the feasible set yields a lower bound on the optimal value, which can be used as a stopping criterion and as a certificate of approximation quality at every iteration. The lower bound is given by

$$v_t = \min_{s \in \mathcal{D}} [f(x^{(t)}) + \langle \nabla f(x^{(t)}), s - x^{(t)} \rangle],$$

and the **Frank-Wolfe gap** is defined as:

$$f(x^{(t)}) - v_t.$$

2.3.3 Convergence Analysis

Our objective function is convex and continuously differentiable, since the matrix Q is positive semidefinite. Moreover, the feasible set D is compact and convex, being the Cartesian product of simplices.

Under these assumptions, the classical Frank–Wolfe algorithm is known to converge sublinearly. More precisely, if $x^{(t)}$ denotes the iterate at iteration t , then the following bound holds [1]:

$$f(x^{(t)}) - f(x^*) \leq \mathcal{O}\left(\frac{1}{t}\right),$$

where x^* is an optimal solution of the problem.

An equivalent convergence rate holds for the Frank–Wolfe gap, defined as

$$g(x^{(t)}) = \max_{s \in D} \langle \nabla f(x^{(t)}), x^{(t)} - s \rangle,$$

which provides a computable certificate of suboptimality and can be used as a stopping criterion.

Curvature Constant. More specifically, the convergence analysis of the Frank–Wolfe algorithm relies on a measure of the nonlinearity of the objective function over the feasible set. Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be a convex and continuously differentiable function, and let $\mathcal{D} \subset \mathbb{R}^n$ be compact and convex. The *curvature constant* of f over $\mathcal{D}$ is defined as

$$C_f := \sup_{\substack{x, s \in \mathcal{D} \\ \gamma \in (0, 1]}} \frac{2}{\gamma^2} (f(x + \gamma(s - x)) - f(x) - \gamma \langle \nabla f(x), s - x \rangle). \quad (1)$$

The above definition follows [1].

Theorem - Primal Convergence. *For each $k \geq 1$, the iterates $x^{(k)}$ of the algorithm satisfy*

$$f(x^{(k)}) - f(x^*) \leq \frac{2C_f}{k+2}(1 + \delta),$$

where $x^* \in D$ is an optimal solution to the problem, and $\delta \geq 0$ is the accuracy to which the internal linear subproblems are solved.

Theorem proof is available at appendix A in [1].

Specialization to the Quadratic Case. For our objective $f(x) = x^\top Qx + q^\top x$, the Hessian is constant:

$$\nabla^2 f(x) = 2Q.$$

Hence the gradient is Lipschitz continuous with respect to the Euclidean norm with constant

$$L = 2\lambda_{\max}(Q).$$

Following [1], a standard bound on the curvature constant yields

$$C_f \leq L \operatorname{diam}(D)^2 = 2\lambda_{\max}(Q) \operatorname{diam}(D)^2.$$

Since D is the Cartesian product of K simplices, we compute its diameter explicitly. The diameter of a standard simplex is $\sqrt{2}$, as the maximum distance is attained between two distinct vertices e_i and e_j , for which $\|e_i - e_j\|_2^2 = 2$. Because squared Euclidean distances add across blocks, we obtain

$$\operatorname{diam}(D)^2 = \sum_{k=1}^K 2 = 2K.$$

Therefore,

$$C_f \leq 4K\lambda_{\max}(Q).$$

Substituting into the primal convergence bound gives the fully explicit estimate

$$f(x^{(t)}) - f(x^*) \leq \frac{2C_f}{t+2} \leq \frac{8K\lambda_{\max}(Q)}{t+2}.$$

This means that the worst-case convergence rate depends linearly on the number of simplices K and on the largest eigenvalue of Q . In particular, large spectral norm (large $\lambda_{\max}(Q)$) increases the curvature constant and therefore worsens the theoretical rate. Also to be noted that in our implementation, the linear subproblems are solved exactly, hence $\delta = 0$.

3 Data Characterization

In this chapter, we describe the datasets used to evaluate the performance of the proposed algorithms.

3.1 Frank–Wolfe Data Generation

For the implementation and testing of the conditional gradient (Frank–Wolfe) method, we generate synthetic datasets under two different conditioning scenarios. The goal is to evaluate how the algorithm’s performance is affected by the geometry and conditioning of the objective function. We will consider two settings, the first having a well-conditioned Q , while the second having an ill-conditioned one.

3.1.1 Matrix Generation

We generate the quadratic matrix Q through a spectral decomposition of the form

$$Q = U\Lambda U^T,$$

where U is a randomly generated orthogonal matrix and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ is a diagonal matrix containing the eigenvalues of Q . By construction, Q is symmetric positive semidefinite. The distribution of the eigenvalues λ_i allows us to explicitly control the conditioning of the problem.

For the well-conditioned setting, the eigenvalues are sampled uniformly in the interval $\lambda_i \in [0, 10]$, resulting in a moderate condition number. While for the ill-conditioned setting, the eigenvalues are sampled in the interval $\lambda_i \in [0, 10^4]$ leading to a significantly larger condition number and therefore a more challenging optimization landscape.

Expected Results Theoretically, the Frank–Wolfe algorithm converges at a sublinear rate of $\mathcal{O}(1/k)$, with a bound governed by the curvature constant C_f [1]. In the well-conditioned case, the narrow spectrum of Q results in a moderate C_f , leading to a stable, regular decrease of the objective value and a smooth decay of the duality gap with minimal oscillatory behavior. Conversely, ill-conditioning increases C_f and distorts the objective geometry, significantly slowing practical convergence and inducing more pronounced zig-zagging in the iterates. This dense configuration serves as a benchmark for evaluating the algorithm under realistic, fully coupled conditions where no structural advantages can be exploited.

3.1.2 Characterization of the Linear Term

To investigate how the geometry of the solution affects the practical behavior of the Conditional Gradient method, we construct the linear

term q so as to control the location of the unconstrained minimizer. For our quadratic objective the unconstrained minimum x_u satisfies

$$\nabla f(x_u) = 2Qx_u + q = 0,$$

and therefore, for any prescribed vector $x_u \in \mathbb{R}^n$, we can define

$$q = -2Qx_u.$$

This construction allows us to prescribe the location of the stationary point of the quadratic function and to study how its relative position with respect to the feasible set D influences the structure of the optimal solution x^* .

In the **first scenario**, x_u is chosen in the interior of D , meaning that for every block $k \in K$ we have

$$\sum_{i \in I_k} (x_u)_i = 1 \quad \text{and} \quad (x_u)_i > 0.$$

In this case, the unconstrained minimizer is feasible. If Q is positive definite, the minimizer is unique and therefore coincides with the global solution x^* . If instead Q is only positive semidefinite, the quadratic form may have a nontrivial null space and the set of unconstrained minimizers need not be a singleton. In that situation, x_u is one stationary point satisfying $\nabla f(x_u) = 0$, but other minimizers may exist. In either case, the optimal solution lies in the relative interior of D , and no non-negativity constraint is active.

In the **second scenario**, x_u is chosen so that it significantly violates the non-negativity constraints in many components. The projection of x_u onto D then lies on a low-dimensional face of the feasible set, typically corresponding to vertices or sparse extreme-point configurations. In this setting, the optimal solution x^* has a large active set, with many coordinates equal to zero. This configuration emphasizes the role of vertex selection in the linear minimization oracle.

In the **third scenario**, x_u satisfies the block-sum constraints but violates non-negativity only in a small number of components. Consequently, the optimal solution lies on a higher-dimensional face of D , where only a limited subset of constraints is active. In this case, the solution cannot be represented by a single vertex per block and must instead be expressed as a convex combination of multiple extreme points. This geometry is known to induce the characteristic zig-zagging behavior of the Frank–Wolfe algorithm, which may slow practical convergence.

3.2 Dataset Sizes

For both settings we are going to use the same dataset size in the order of 10^4 . This means that the size of our matrix Q is $10^4 \times 10^4$, thus the memory requirement for storing such matrix is in the order of $O(n^2)$, and by using double precision (8 byte per element) the matrix would take up to 800 MB of RAM.

$$10^8 \times 8 = 800MB$$

4 Conditional Gradient Implementation

4.1 Code Snippets

4.2 Convergence And Evaluation

5 General-Purpose Solver Implementation (A2)

To solve the problem using a general-purpose solver as required by the project assignment, we chose Gurobi Optimizer, interfacing with it via its Python API (`gurobipy`).

The mathematical formulation of the problem was mapped to the solver's structures as follows:

- **Variables and Bounds:** We defined a vector of continuous variables x of size n . The non-negativity constraints $x \geq 0$ are natively handled by setting the lower bound parameter to zero (`lb=0.0`) during variable initialization.
- **Objective Function:** The objective function was formulated directly as $x^T Q x + q^T x$, perfectly matching the project's requirements without scaling factors.
- **Equality Constraints:** The disjoint simplices constraints $\sum_{i \in I^k} x_i = 1$ for $k \in K$ were expressed in matrix form as $Ax = \mathbf{1}$. To drastically optimize memory consumption and handle large-scale datasets efficiently, the matrix A was constructed and passed to the solver as a compressed sparse row matrix (`scipy.sparse.csr_matrix`).
- **Solver Configuration:** To align the general-purpose solver with the Active-Set theoretical framework, we explicitly forced Gurobi to

use the Primal Simplex algorithm by setting the model parameter `Method = 0`.

5.1 Code Snippets

5.2 Convergence And Evaluation

6 Conclusions

References

- [1] Martin Jaggi. Revisiting frank-wolfe: Projection-free sparse convex optimization. In *Proceedings of the 30th International Conference on Machine Learning*, 2013.